Human-in-the-Loop Approval for Automated Claim Decisions

Scoping note — should you fund an approval control layer before automating claim decisions?

The problem

You want to automate routine claim decisions. The obstacle is not whether the AI is accurate — it is that you cannot tell *which* decisions it got wrong until after the money has left.

An agent that is right 94% of the time is not 94% good enough when the remaining 6% includes a $40,000 payout on a policy that lapsed last month. What you need is not a smarter agent. It is a reliable boundary: the agent handles what it should, a human sees everything else, and that human gets enough context to decide in under two minutes.

What production-ready means here

- Thresholds — claim value, model confidence, policy exceptions — enforced **in code**, not written into the AI's instructions.
- Every paused claim arrives with the reasoning, the policy clauses cited, and the claimant's history attached.
- Defined behaviour when nobody responds: escalation path and timeout, not indefinite waiting.
- An immutable record of what was proposed, what was approved, by whom, and why — durable enough for a regulator.
- Resume that re-validates before acting, and cannot double-pay if triggered twice.

What breaks in a demo version. The approval is implemented as an instruction — "ask a human if you are unsure" — and models do not reliably obey instructions. The queue has no service-level target, so claims age silently. Approvers get a yes/no button with no reasoning attached, so they approve everything — which is worse than no review, because now there is a signature on it. And the audit trail is application logs, which will not survive an examination.

Three risks to weigh before committing budget

1. **Rubber-stamping.** If 90%+ of paused claims are approved unchanged, you have a speed bump, not a control. We track override rate as a primary KPI from week one; a rate below roughly 5% means the thresholds are miscalibrated, not that the AI is good.
2. **Throughput versus safety.** Every tightened threshold buys safety with adjuster time. Pausing 30% of claims can cost more than the automation saves. We agree a target pause rate before we build, not after.
3. **Confidence scores are not calibrated out of the box.** A model's self-reported confidence correlates only loosely with whether it is right. Gating on an uncalibrated number is gating on nothing — which is why calibration is week one, not an afterthought.

Scope and timeline — 2-week engagement

	Focus	Deliverables
Week 1	Calibration & controls	Confidence calibrated against ~100 labelled historical claims; threshold policy engine; durable pause/resume; immutable audit log
Week 2	Operations & handover	Approval queue interface; timeout and escalation rules; idempotent resume; override-rate dashboard; one claim type live end-to-end; load test; runbook and engineering handover

Roughly 60% of the effort is the control and audit layer, 25% integration into your existing claims flow, and 15% calibration and testing. **One dependency on you:** access to about 100 historically adjudicated claims in week one. Without them we can ship the mechanism but not tune the thresholds, and you would be gating on a number that does not yet mean anything.

Deliberately out of scope for v1, so that it ships working: one claim type and a single approver (no routing by category, seniority or region); web queue only, no mobile or email approvals; static thresholds, tuned once at launch; new claims only, with existing open claims not migrated; single-region deployment with no active failover.

Phase 2 — multi-approver routing and delegation, thresholds that retune automatically from override history, mobile and email approvals, additional claim types, and a regulator-facing audit export. We recommend scoping this from your first 30 days of production override data rather than guessing at it now: that month tells us which of these you actually need.